Aim

To implement a Hidden Markov Model (HMM) from scratch for Part-of-Speech (POS) tagging on a contractual legal corpus, and to use the Viterbi dynamic-programming decoding algorithm to predict the optimal POS-tag sequence for an unseen test sentence, evaluating the result against NLTK's reference tagger.

Theory

A Hidden Markov Model (HMM) is a generative probabilistic sequence classifier in which Part-of-Speech tags act as **hidden states** ($Q = \{q_1, \ldots, q_N\}$) and words are the visible **observations** ($O = \{o_1, \ldots, o_T\}$). An HMM is characterised by three parameters, $\lambda = (\pi, A, B)$:

- **Initial state probability vector (π):** $\pi_i = \mathrm{P}(q_i \text{ at } t = 1)$, the probability that a sentence begins with tag q_i.
- **Transition probability matrix (A):** the probability of moving between consecutive tags under the 1st-order Markov assumption.

$$a_{ij} = \mathrm{P}(t_k = q_j \mid t_{k-1} = q_i) = \mathrm{Count}(q_i \rightarrow q_j) \,/\, \mathrm{Count}(q_i)$$

- **Emission probability matrix (B):** the probability that hidden state q_j generates the surface token o_k.

$$b_j(k) = \mathrm{P}(o_k \mid q_j) = \mathrm{Count}(o_k, q_j) \,/\, \mathrm{Count}(q_j)$$

To decode the hidden tag sequence, the **Viterbi algorithm** uses dynamic programming to maximise path probabilities recursively:

$$V_t(j) = \max_{i=1\ldots N} [\, V_{t-1}(i) \cdot a_{ij} \,] \cdot b_j(o_t)$$

Backpointers, backpointer(t, j) = argmax_i [$V_{t-1}(i) \cdot a_{ij}$], are recorded at each time step, and the globally optimal tag sequence is reconstructed by backtracking from the highest-probability state at the final token T.

Experimental Setup & Implementation Details

Domain case study	Terms & Conditions Summarizer (Amazon & Alibaba legal agreements)

Training corpus	10 domain sentences, e.g. *“Amazon provides reliable cloud services”*, *“Users accept legal terms and conditions”*, *“The company disclaims all express warranties”*
States (tags)	N = 11 unique POS tags: DT, IN, JJ, NN, NNP, NNS, RB, TO, VB, VBP, VBZ
Vocabulary	M = 34 unique words
Test sentence (unseen)	*“Customers accept all amended terms”* (T = 5 words)

Execution Results & Tables

1. Transition Probability Matrix (A) — Excerpt

From \ To	NNS	VBP	DT	JJ	NN
NNS (Plural noun)	0.002	**0.551**	0.002	0.002	0.002
VBP (Verb, non-3rd)	0.002	0.002	**0.395**	0.264	0.002
DT (Determiner)	0.002	0.002	0.002	**0.468**	0.313
JJ (Adjective)	**0.468**	0.002	0.002	0.002	**0.468**

Highlighted cells mark the dominant transition(s) out of each tag.

2. Viterbi Trellis Computation & Backtracking

t	Word	Recursion	Score	State
1	*Customers*	$\pi_{NNS} \cdot P(\text{“Customers”} \mid \text{NNS}) = 0.20 \times 0.224$	0.0448	NNS
2	*accept*	$V_1(\text{NNS}) \cdot a_{\text{NNS}\to\text{VBP}} \cdot P(\text{“accept”} \mid \text{VBP}) = 0.0448 \times 0.551 \times 0.252$	0.00622	VBP
3	*all*	$V_2(\text{VBP}) \cdot a_{\text{VBP}\to\text{DT}} \cdot P(\text{“all”} \mid \text{DT}) = 0.00622 \times 0.395 \times 0.362$	0.000889	DT
4	*amended*	$V_3(\text{DT}) \cdot a_{\text{DT}\to\text{JJ}} \cdot P(\text{“amended”} \mid \text{JJ}) = 0.000889 \times 0.468 \times 0.165$	0.0000686	JJ

5	*terms*	$V_4(JJ) \cdot a_{JJ \to NNS} \cdot P(\text{“terms”} \mid NNS) = 0.0000686 \times 0.468 \times 0.224$	0.0000072	NNS

3. Verification with NLTK Reference Tagger

Word	HMM Viterbi Tag	NLTK Reference Tag	Match
Customers	NNS	NNS	**YES ✓**
accept	VBP	VBP	**YES ✓**
all	DT	DT	**YES ✓**
amended	JJ	JJ	**YES ✓**
terms	NNS	NNS	**YES ✓**

Final predicted sequence:	Customers/NNS accept/VBP all/DT amended/JJ terms/NNS
Tagging accuracy:	**100.0%** (5/5 matching tags)

Conclusion & Observations

1. Mechanism of HMM POS Prediction

An HMM assigns tags by balancing *syntactic sequence grammar* (how natural the tag transition $t_i \to t_j$ is) against *lexical emission likelihood* (how likely the word w_t is given tag t_j). The Viterbi algorithm prunes suboptimal paths at every step, reducing the complexity from exponential, O(N^T), to O($T \cdot N^2$).

2. Observations on Probabilities

- The transition matrix mirrors English legal grammar: plural nouns (NNS) transition to present-tense verbs (VBP) with the highest probability (≈ 0.55), and determiners (DT) predominantly precede adjectives (JJ) and nouns (NN).
- Emission probabilities accurately resolve polysemous words based on their relative frequency in the domain corpus.

3. Limitations & Difficulties Observed

- **Out-of-vocabulary (OOV) bottleneck:** if an unseen word is absent from the training dictionary, its emission probability collapses to 0 unless Laplace smoothing is applied.
- **Markov horizon limit:** a 1st-order HMM only considers the immediately preceding tag (t_{k-1}), ignoring long-range contractual context across compound clauses.

Google Colab Implementation

- **Google Colab Link:** Experiment_HMM_POS_Tagging.ipynb